Fiber Laser Z-Axis Height Control

Eliminating rubber-band oscillation with fixed-point PD control, hysteresis, and slew-rate limiting

---

1. The Problem

The fiber laser uses a magnetic field sensor to read a frequency value correlated to the head's distance from the material. A target frequency is calibrated once at the start of each cut sequence, and a clock reads the sensor 256 times per second. Because the material's surface is not perfectly flat, the real distance drifts as the head moves across X and Y, so the Z-axis motor must continuously correct height to hold the target frequency.

Original approach and its failure

The original controller used bang-bang logic: if the reading was too high, command the motor down; if too low, command it up, always with the same-size correction. In practice this caused the laser to move toward the correct position, overshoot it, then immediately compute a correction in the opposite direction (slightly smaller), producing a fast back-and-forth "rubber-banding" motion.

2. Why It Rubber-Bands: Three Root Causes

1. No hysteresis. A single on/off threshold means any sensor noise sitting right at that line flips the direction command every sample. At 256 Hz that reads as constant chatter.

2. No damping (derivative term). The correction only reacts to the current error, not to how fast that error is already closing. The controller keeps pushing full-strength right up to the target, then overshoots because of sensor and motor lag.

3. No slew-rate limiting. Nothing stops the commanded output from swinging from a large positive value to a large negative value in a single tick — an instantaneous direction flip.

3. Fix #1 — Hysteresis Deadband

A deadband is a "do nothing" zone around the target. The key insight is that two thresholds are needed, not one — this is hysteresis:

- **deadband_out** (outer boundary): correction starts only once the error exceeds this value.
- **deadband_in** (inner boundary, smaller than deadband_out): correction stops only once the error falls back inside this tighter band.
- The gap between the two thresholds absorbs sensor noise — a value sitting right at a single line can no longer cause chatter.
- Set deadband_out slightly above the sensor's noise floor at rest (measure the sensor stationary over flat material and observe the spread of raw values).
- deadband_in is typically about half of deadband_out.

Pseudocode:

```
if not correcting:
    if abs_error > deadband_out: start correcting
else:
    if abs_error < deadband_in: stop correcting, hold
```

4. Fix #2 — Proportional-Derivative (PD) Control

Kp — Proportional Gain

```
P term = Kp × error
```

- Reacts to how far off the current reading is, right now.
- Bigger error produces a bigger correction, and the correction shrinks automatically as the error shrinks — this alone is a major improvement over bang-bang control.
- Too small a Kp: sluggish response to real height changes.
- Too large a Kp: overshoot, because a large error times a large gain produces a correction big enough to fly past the target before the next sample can react.

Kd — Derivative Gain

```
D term = Kd × (error_now − error_previous)
```

- Reacts to how fast the error is changing, not just its current size.
- If the error is shrinking quickly (approaching the target fast), this term subtracts from the correction — easing off before arrival rather than after overshooting.
- This is the single most important fix for lag-driven overshoot and rubber-banding.
- Rule of thumb: start Kd at roughly 2-3x Kp, since real physical lag (sensor delay, motor response, mechanical settling) needs strong damping to counteract it.
- Note: the integral ("I") term from full PID was intentionally omitted. Integral corrects small steady-state offsets over time but adds tuning complexity (windup risk) that isn't needed for this fast height-following problem.

5. Fix #3 — Slew-Rate Limiting

This is separate from Kp/Kd and solves a different problem: even a well-tuned PD controller can occasionally compute a large jump in output between two samples. The slew limiter caps how much the commanded output is allowed to change per sample, regardless of what the raw PD math produces.

```
if (new_output − last_output) > max_change:
    new_output = last_output + max_change
```

This directly prevents the "moves correctly, then immediately snaps to the opposite direction" behavior — the output is forced to ramp toward a new value smoothly rather than teleport there. It functions as a speed limit on the controller's decisions, layered on top of the separate speed limit (max_step) on the

motor's physical motion.

6. Integer-Only Math: Q16.16 Fixed Point

Floating point is avoided here because it is slower and less predictable on many embedded targets. The standard alternative is Q-format fixed point: represent a fractional number as a plain integer pre-multiplied by a fixed power of two. In "Q16.16", every value is shifted left by 16 bits before storage, so the bottom 16 bits represent the fractional part.

Building a constant — to_q()

```
to_q(0, 3, 10) = (0 << 16) + (3 << 16) / 10 = 19661  // represents 0.3
```

Multiplying two Q16.16 values — q_mul()

```
int64_t product = (int64_t)a * b;
return product >> 16;
```

Multiplying two values each scaled by 65536 produces a result scaled by 65536^2, so the result must be shifted right by 16 afterward to bring it back to a single scale factor. Widening to int64_t before the shift is important — multiplying two 32-bit Q16.16 values can overflow a 32-bit register before the shift-back.

7. A Real Bug Found During Testing

When the controller was ported to C and compiled, the motor step output flatlined at zero on every sample. The cause: the slew limit was set smaller than 1.0 in Q16.16 (less than 65536), and the output accumulator was being re-quantized down to the truncated integer step every cycle. This meant the fractional progress had nowhere to accumulate, so the output stayed stuck below the threshold needed to produce even a single motor pulse — the controller silently did nothing, indefinitely.

The fix: keep the accumulator (last_output_q) at full Q16.16 precision across samples, and only truncate to an integer at the very last step, right before handing the value to the motor. This allows even a slow ramp rate to genuinely creep upward sample by sample until it crosses an integer boundary — the standard technique for representing sub-unit rates of change in integer-only math.

Verified by compiling with `gcc -Wall -Wextra -std=c99 -O2` (zero warnings) and confirming convergence with a traced test run.

8. Verified Test Results

Starting from a 40-unit sensor error, the corrected controller converges smoothly into the deadband with no overshoot and no oscillation:

Sample	Error	Step	Notes
0	40	+2	Correction begins
2	27	+6	Peak correction (largest error)

9	10	+1	Tapering as error shrinks
18	3	+1	Entering inner deadband
20+	3	0	Settled — holding, zero output

Result: converges from a 40-unit error to inside the deadband in approximately 20 samples (~80 ms at 256 Hz). No overshoot, no reversal, no chatter.

9. Sampling Rate In Context

Commercial capacitive height controllers used on fiber laser cutting heads (e.g. the widely-used BCS100) are specified at a sampling rate of 1,000 times per second (1 kHz) — this is the rate at which a fresh distance measurement becomes available, and in these systems the sampling rate effectively is the control loop rate, since a new motor correction is computed each time a new reading arrives.

	This system	Commercial (e.g. BCS100)
Sample rate	256 Hz	1,000 Hz
Time between samples	~3.9 ms	~1 ms
Loop architecture	Measurement = control decision, every cycle	Same architecture, faster cycle

Takeaway: the slower 256 Hz loop makes Kd tuning and slew-rate limiting even more important, since roughly 4x more time passes between corrections, giving more opportunity for real-world drift to accumulate.

10. Handling Sensor Noise (Recommended Next Step)

At any sample rate, sensor noise is inevitable. This creates a known problem called **derivative kick**: the Kd term computes a difference between two consecutive readings, and if that difference is mostly noise rather than real motion, the derivative term amplifies the noise instead of smoothing it out.

The standard fix is to low-pass filter the raw sensor reading before it reaches the controller. The simplest integer-only version is an exponential moving average (EMA), which can be implemented as a single line using only subtraction, a bit shift, and addition:

```
filtered += (raw – filtered) >> N
```

- Larger N: heavier smoothing, better noise rejection, slower reaction to real changes.
- Smaller N: lighter smoothing, faster reaction, more noise passes through.
- Insert as a preprocessing step: read raw frequency, run through the EMA filter, feed the smoothed value into the existing controller unchanged.

11. The Full Control Pipeline

Executed once per sample, at 256 Hz:

Step	Action	Purpose
1. Read	Raw frequency from magnetic sensor	Get current distance measurement
2. Filter	EMA smoothing (recommended)	Remove noise before it reaches Kd
3. Compare	Compute error vs. calibrated target	Establish how far off height is
4. Deadband	Hysteresis check	Decide whether to act at all
5. PD Math	Kp × error + Kd × Δerror (Q16.16)	Compute a scaled, damped correction
6. Slew Limit	Cap change in commanded output	Prevent instant direction reversal
7. Clamp	Hard max_step ceiling	Absolute safety limit
8. Command	Truncate to integer, pulse motor	Physically move the Z-axis

12. Key Takeaways

- Bang-bang control oscillates whenever there is real-world lag — sensor delay, motor response, mechanical settling.
- Hysteresis deadband kills threshold chatter by requiring a full crossing of two separate thresholds, not a single line.
- Proportional gain (Kp) scales correction to error size instead of always pushing full-strength.
- Derivative gain (Kd) is the key fix for overshoot — it brakes before arrival, not after.
- Slew-rate limiting prevents instant direction reversals regardless of what the PD math computes.
- Q16.16 fixed-point delivers real fractional precision using only integer shifts and multiplies — no floating point required.
- Solution verified: compiles clean, converges smoothly from a 40-unit error to settled in ~80 ms, with zero oscillation.